Advanced Programming Techniques

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

C++ Inheritance

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

What a mess

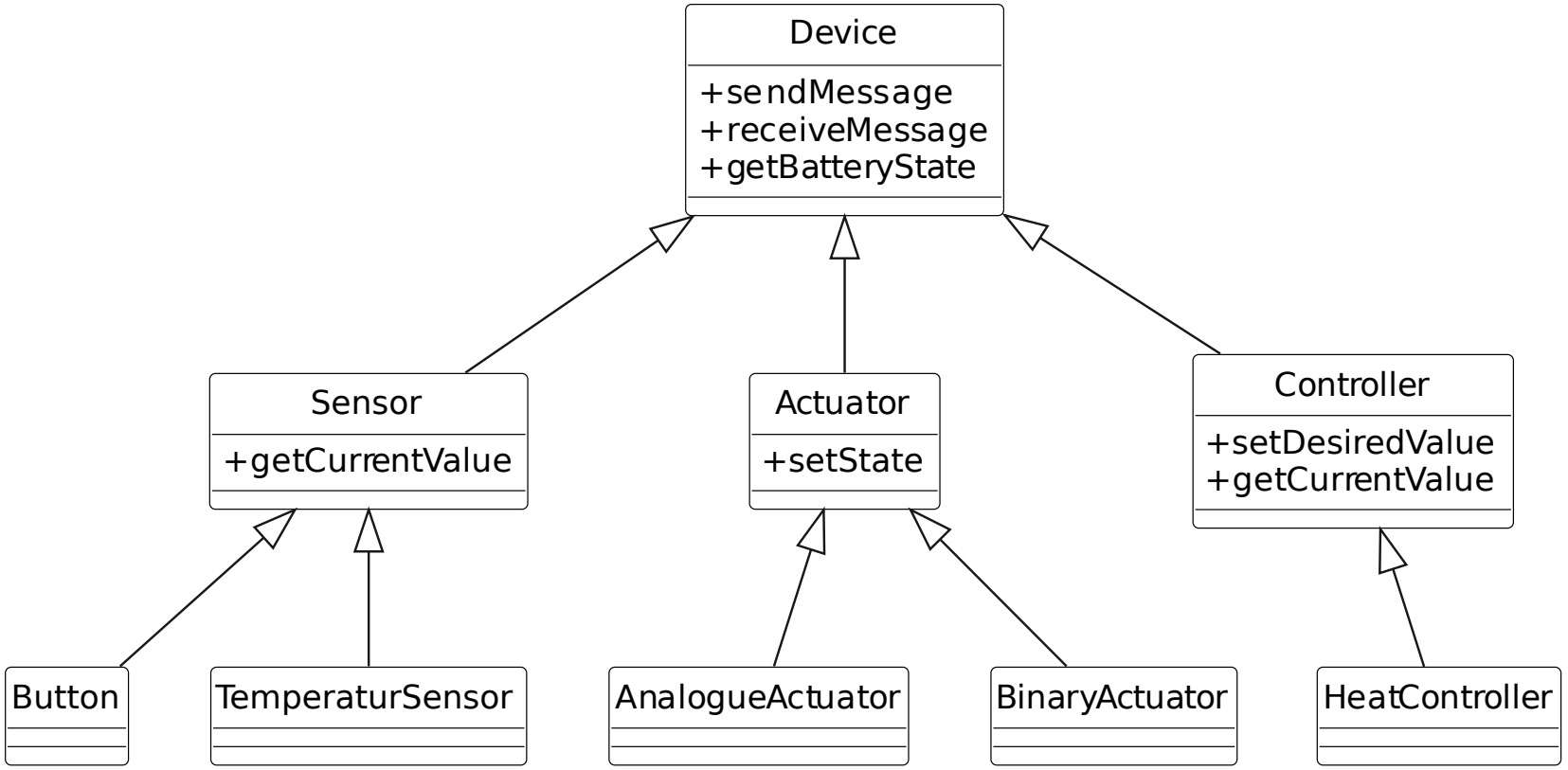
- Remember the heat controller?
- There's more in home automation!

Roller Shutter

Images Sources: https://shop.zwave.eu, https://www.fibaroaustria.at

Order!

- **Humans order things by categorizing**



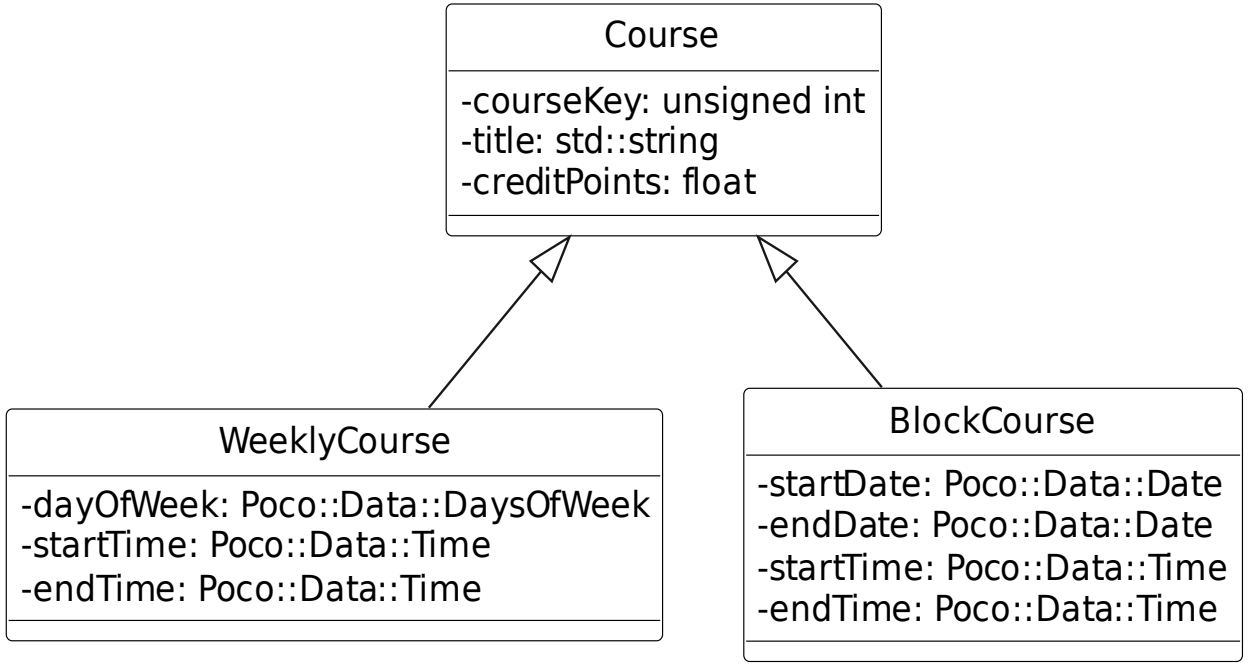
Inheritance (“is-a” relationship)

- **From Wikipedia:**
  - Inheritance is defined as deriving new classes (sub classes) from existing ones (super class or base class) and forming them into a hierarchy of classes.
  - In most class-based object-oriented languages, an object created through inheritance (a "child object") acquires all the properties and behaviors of the parent object (except: constructors, destructor, overloaded operators and friend functions of the base class).
  - Inheritance allows programmers
    - to create classes that are built upon existing classes,
    - to specify a new implementation while maintaining the same behaviors ,
    - to independently extend original software via public classes and interfaces.

Another example

- **Weekly courses and block courses**
  - Basically same data
  - Specification of event date/time differs

Live Coding



Defining derived classes

- **`Class Derived: public Base {…}`**
- Example

```
class WeeklyCourse: public Course {
private:
    Poco::DateTime::DaysOfWeek dayOfWeek;
    Poco::Data::Time startTime;
    Poco::Data::Time endTime;

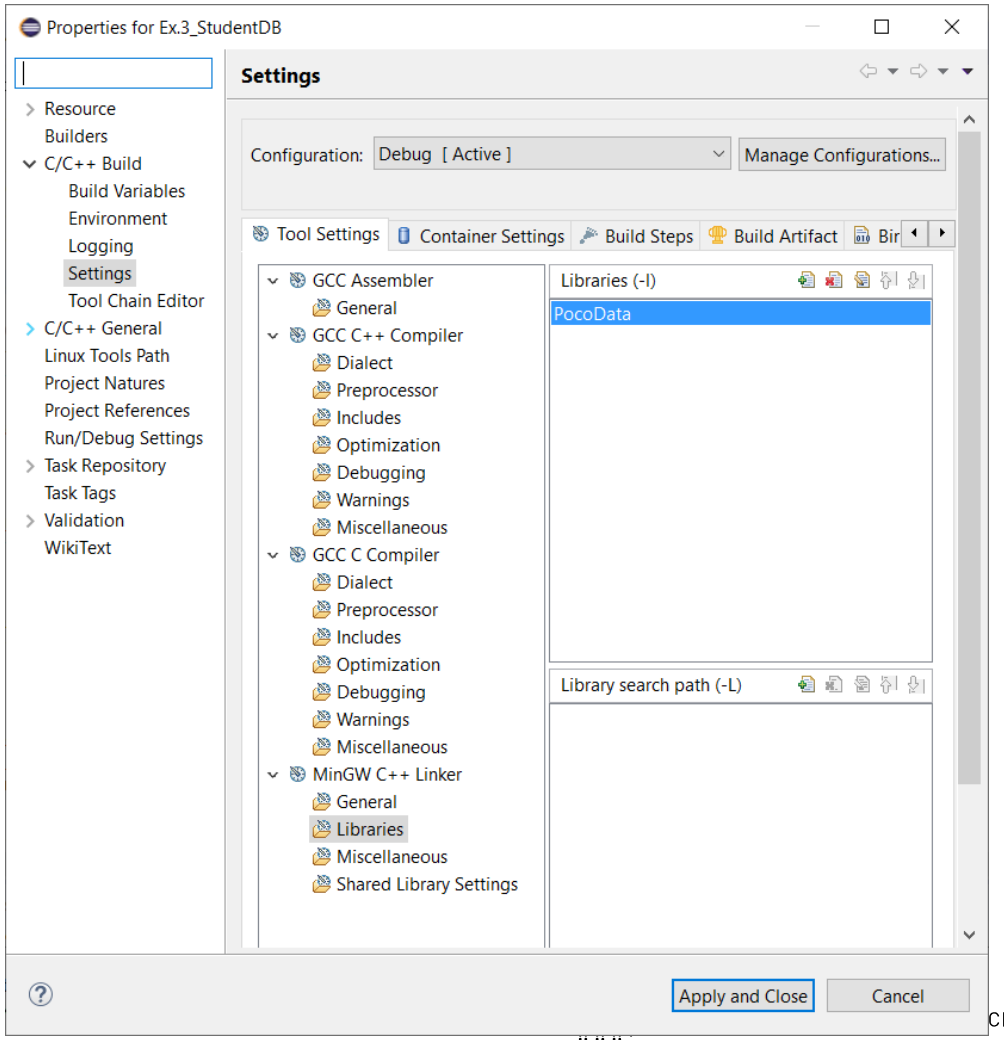
public:
    WeeklyCourse(unsigned int courseKey, std::string title, std::string major,
            float creditPoints, Poco::DateTime::DaysOfWeek dayOfWeek,
            Poco::Data::Time startTime, Poco::Data::Time endTime);
};
```

`Date`, `Time`, `DateTime` from Poco

- **`Timestamp`** represents the monotonic time
  - Use TimeVal (64-bit integer) for space eficient storage
- **`DateTime`** allows access to hour, minute, second, day-of-week, day-of-month, month, year based on the Gregorian calendar
- **`Date`** and **`Time`** are wrappers that represent only the parts
- Documentation online, start with https://pocoproject.org/docs/Poco.DateTime.html

Date, Time, DateTime from Poco

- **Poco usage**
  - Requires the addition of libraries for the Poco package used
  - Add libraries as shown in the previous slides
  - Using date/time requires “PocoData”
  - Using networking (upcoming lecture) requires “PocoNet”
  - Using multi-threading (upcoming lecture) requires “pthread”



Implementing a constructor

- Pass parameter values as arguments to base class constructor

```
WeeklyCourse::WeeklyCourse(unsigned int courseKey, std::string title,
        std::string major,
        float creditPoints, Poco::DateTime::DaysOfWeek dayOfWeek,
        Poco::Data::Time startTime, Poco::Data::Time endTime)
    : Course(courseKey, title, major, creditPoints),
      dayOfWeek{dayOfWeek}, startTime{startTime}, endTime{endTime} {
}
```

Inherited methods (1)

- **Inherited methods can be invoked as if they were the derived class's own methods**
  - ```
    WeeklyCourse apt(42, "APT", "Automation", 5,
                     Poco::DateTime::DaysOfWeek::WEDNESDAY,
                     Poco::Data::Time(17, 45, 0),
                     Poco::Data::Time(19, 15, 0));
    apt.print();
    ```
  - → `course 42: APT, for major Automation`
  - Some information is missing
- **If the inherited method does not match the requirements of the derived class, it can be overridden**
  - Simply define it again for the derived class

Live Coding
    ```

Inherited methods (2)

- **A base class can define “protected” members**
  - These members can only be accessed from the methods of the class itself or derived classes (i. e. they cannot be accessed from “outside” as public methods can)
  - In the UML class diagram the “protected” visibility is indicated by a sharp (“#”)

Subtyping

- **In C++, a derived class is also a subtype of the base class**
  - Objects of subtypes can be used where objects of base type are expected (subtype polymorphism)

```
void f(Course& course) {
    course.print();
}

int main (void){
    cout << "StudentDb started." << endl << endl;
    WeeklyCourse apt(42, "APT", "Automation", 5,
        Poco::DateTime::DaysOfWeek::WEDNESDAY,
        Poco::Data::Time(17, 45, 0), Poco::Data::Time(19, 15, 0));
    f(apt);
    return 0;
}
```

  - → But we still get print result from base class

Static vs. dynamic polymorphism

- **Static polymorphism**
  - Compiler accepts object of derived class, but assumes it to be of declared type and invokes declared type's method
- **Dynamic polymorphism**
  - Compiler generates code that checks at runtime what the type of the object is and invokes appropriate method
  - Dynamic polymorphism selected with keyword virtual

```
virtual void print();
```

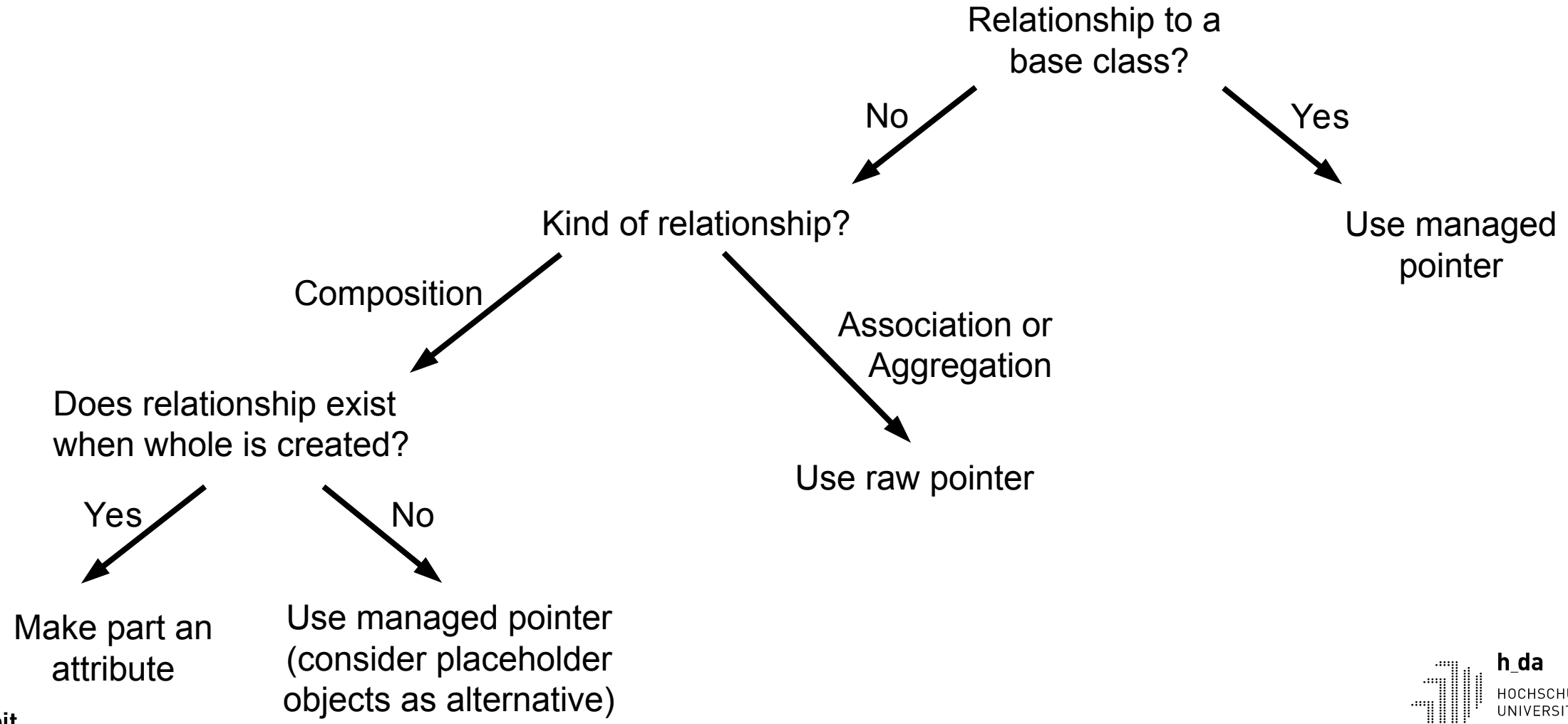
  - Especially important for destructor (else wrong destructor is invoked)
- **Check type at run-time (rarely used)**
  - `dynamic_cast<type>(pointer)` → `nullptr` if not pointing to type

What about the list of courses?

- **We have list<Course>**
- **Inserting “reduces” derived class to base class**
- **We cannot store objects of derived classes, because they use more memory**
  - → Allocate space on heap (as much memory as required) and store pointer
- **We still need composition relationship!**
  - Possible solution: destructor of `Course` frees objects in list. Error prone, don’t!
  - Use `unique_ptr` from standard library

Live Coding

Guidelines for implementing relationships



Abstract types

- **A virtual method can be defined as "0"**

```
class Course {
    // …
public:
    // …
    virtual void print() const = 0;
};
```

  - Such a method is called a "pure virtual method"
- **A class with such a method is an abstract type (or class) (vs. the concrete types (or classes that we defined up to now)**
  - No instances can be created (because not all methods have implementations)
  - The class can only be used as base class